Apuntes – Persistencia local con Room

Desarrollo de Aplicaciones I · UADE · Guía de Ejercicios: Persistencia local con Room (v1.0)

Proyecto: `EJERCICIOS-KOTLIN-ROOM` · Felipe Forte · Kotlin + Jetpack Compose + Room 2.8.5 (KSP)

Cómo leer estos apuntes: cada ejercicio tiene un recuadro **“En pocas palabras”** (qué se hace y por qué), uno de **“Aclaraciones para novatos”** (lo que suele confundir), la **explicación con el código real** del proyecto, las **pruebas hechas en el emulador** (con capturas y consultas a la base) y las **respuestas a las preguntas de la guía**. Todas las consultas a la base se hicieron copiando el archivo SQLite del emulador y consultándolo con `sqlite3`: muestran lo que realmente quedó guardado.

Glosario para empezar

Término	Qué significa, en simple
Persistir	Guardar un dato de forma que sobreviva a cerrar la app, matar el proceso o reiniciar el teléfono.
SQLite	El motor de base de datos que viene en Android. Guarda todo en **un archivo** dentro de la app (por ejemplo `notas.db`).
Tabla / fila / columna	Una tabla es como una planilla: cada *fila* es un registro (una nota) y cada *columna* un dato (título, fecha).
SQL	El idioma para hablarle a la base: `SELECT` (leer), `INSERT` (agregar), `UPDATE` (modificar), `DELETE` (borrar).
Room	Biblioteca de Android que se pone encima de SQLite: uno escribe clases de Kotlin con anotaciones y Room genera el código que habla con SQLite.
Entity	Una `data class` con `@Entity`: describe **una tabla**. Cada objeto = una fila.
DAO	Interface con `@Dao`: lista las **operaciones** sobre la tabla (insertar, buscar, borrar). Room escribe la implementación.
RoomDatabase	La clase que representa **la base entera**: qué tablas tiene, qué versión es, y da acceso a los DAO.
Clave primaria (PK)	La columna que identifica cada fila sin repetirse (acá, `id`).
Clave foránea (FK)	Una columna que apunta a la PK de otra tabla (una nota "pertenece" a una categoría).
Índice	Estructura extra que acelera buscar u ordenar por una columna, y puede impedir repetidos (índice único).
Esquema	La "forma" de la base: qué tablas y columnas tiene. Si cambia, hay que **migrar**.
Migración	Instrucciones para pasar una base instalada de una versión del esquema a la siguiente **sin perder datos**.
Transacción	Grupo de operaciones que se hacen todas o ninguna.
KSP	Herramienta que, al compilar, lee las anotaciones de Room y genera el código. Si el SQL está mal, **no compila**.
suspend / corrutina	Las operaciones de base pueden tardar: se marcan `suspend` y se llaman desde una corrutina, sin congelar la pantalla.
Flow	Un "canal" de valores. Un DAO que devuelve `Flow` vuelve a emitir la lista cada vez que la tabla cambia.
Database Inspector	Herramienta de Android Studio (App Inspection) para ver y editar la base de la app mientras corre en debug.

Organización del proyecto en carpetas

El código se organiza por capas, como en el proyecto de MVVM (y el repositorio de referencia *PokedexAcademy-MVVM-Clean*). En cada rama aparecen solo las carpetas de ese ejercicio.

```
app/src/main/java/com/example/ejercicioskotlinroom/
 ├── MainActivity.kt          # punto de entrada
 ├── NotasApp.kt              # Application: crea la ÚNICA instancia de la base (desde el Ej 4)
 ├── core/theme/              # tema de Compose
 ├── data/
 │    ├── local/              # AppDatabase, Converters, Migrations
 │    │    ├── entity/        # tablas (@Entity) y resultados de relaciones
 │    │    └── dao/           # operaciones (@Dao)
 │    └── repository/         # Repository: la API de datos que ve el ViewModel (desde el Ej 9)
 ├── domain/model/            # enums del dominio (Prioridad, EstadoPrestamo)
 └── presentation/            # una carpeta por pantalla (pantalla + ViewModel)

app/src/androidTest/java/com/example/ejercicioskotlinroom/
 └── data/local/              # tests de DAO, converters y migraciones (corren en el emulador)
app/schemas/                  # JSON del esquema de cada versión (exportSchema = true)
```

> **Diferencia con el proyecto MVVM**
>
> Siguiendo la guía de Room, la pantalla y el ViewModel trabajan directamente con las `Entity` (`NotaEntity`): no hay un modelo de dominio aparte ni casos de uso. Por eso `domain/` solo tiene los enums. Los tests de Room van en `androidTest` (necesitan Android para abrir SQLite) y se corren con `./gradlew :app:connectedDebugAndroidTest`.

Ejercicio 1 — ¿Qué se pierde cuando la app se cierra?

Consigna: Lista editable en Compose **sin Room**. Agregar ítems y probar: rotar, volver atrás, cerrar desde recientes y matar el proceso. Registrar qué sobrevive. Tabla “dato / dónde vive / dónde debería vivir”. Rama: `ej01-estado-en-memoria`

> **En pocas palabras**
> Antes de usar una base de datos hay que ver **el problema**: lo que vive en memoria se pierde. La pantalla tiene la misma lista dos veces, una con `remember` y otra con `rememberSaveable`, para comparar qué sobrevive a cada situación.

> **Aclaraciones para novatos**
> - **`remember` no guarda nada en disco:** solo evita que el valor se pierda cuando Compose redibuja. Al rotar, se pierde.
> - **`rememberSaveable`** sobrevive a la rotación (Android lo guarda en un “paquetito” de estado), pero no a que el usuario cierre la app.
> - **Ninguno de los dos es una base de datos.** Para que algo dure “para siempre” hay que escribirlo en un archivo: eso hace Room.
> - **No todo merece guardarse:** el texto a medio escribir o si un diálogo está abierto son del momento; las notas del usuario sí.

Explicación y pruebas





Antes de rotar: 2 ítems en cada lista Después de rotar: **la de remember quedó vacía**

Caso	remember	rememberSaveable
1) Rotar	❌ se pierde (probado)	✅ sobrevive (probado)
2) Volver atrás y entrar	❌	❌ la Activity se destruye del todo
3) Cerrar desde recientes	❌	❌
4) Matar el proceso desde Android Studio	❌	❌

Dato	Dónde vive ahora	Dónde debería vivir
Texto del TextField (borrador)	remember	rememberSaveable (es efímero)
Lista de elementos	remember / rememberSaveable	**Room**: es información del usuario
Preferencia visual (fondo oscuro)	remember	DataStore (preferencia chica, no una tabla)

Preguntas de la guía

1. ¿remember implica persistencia?
No. Solo conserva el valor mientras la composición vive; si la Activity se destruye, se pierde.

2. ¿Diferencia entre conservar estado ante recomposición y conservar datos al reiniciar el proceso?
Recomponer es Compose volviendo a ejecutar la función con el proceso vivo: alcanza `remember`. Reiniciar el proceso borra toda la memoria: solo sobrevive lo que está en disco (archivo, DataStore, Room).

3. ¿Todos los datos de una pantalla deberían guardarse en una base?
No. Lo que importa al usuario y debe durar (notas) va a la base; lo del momento (borrador, diálogo abierto, scroll) es estado de UI.

4. ¿Qué dato sería absurdo persistir?
El “estoy escribiendo” del TextField a medio tipear o un flag de diálogo abierto: no sirven al día siguiente y ensucian la base.

> **Criterio de finalización**
> ✅ Caso que se conserva (rememberSaveable al rotar) y caso que se pierde (remember al rotar, ambos al cerrar) documentados. Room resuelve otro problema: guardar en disco para que los datos sobrevivan a todo.

Ejercicio 2 — Primera Entity: decidir qué se guarda

Consigna: Crear `NotaEntity` revisando campo por campo, agregar una columna propia y describir la tabla resultante.
Rama: `ej02-entity`

> **En pocas palabras**
> Se define **cómo es una fila** de la tabla `notas` con una `data class`. Todavía no se guarda nada: es el "molde". Se agregó la columna `colorEtiqueta`.

> **Aclaraciones para novatos**
> - **Una instancia de `NotaEntity` = una fila.** Cada propiedad = una columna.
> - **`id = 0` no significa que todas las notas tengan id 0:** con `autoGenerate`, el 0 quiere decir "todavía no tiene"; SQLite asigna 1, 2, 3…
> - **SQLite no tiene Boolean:** Room guarda `true/false` como 1/0.
> - **Guardar datos crudos, no formateados:** la fecha va como número (`Long`) para poder ordenar; el "hace 5 minutos" se arma al mostrar.
> - **Cambiar el nombre de una propiedad cambia el nombre de la columna** y obliga a migrar una base ya instalada (Ej 14).

Explicación y pruebas

```
@Entity(tableName = "notas")
data class NotaEntity(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val titulo: String,
    val contenido: String,
    val importante: Boolean = false,
    val creadaEn: Long,                        // milisegundos desde 1970
    val colorEtiqueta: String = "NINGUNO"     // columna agregada
)
```

Columna	Tipo SQLite	Notas
id	INTEGER PK AUTOINCREMENT	identifica la fila
titulo, contenido	TEXT NOT NULL	lo que escribe el usuario
importante	INTEGER NOT NULL	0 / 1
creadaEn	INTEGER NOT NULL	para ordenar por fecha
colorEtiqueta	TEXT NOT NULL	propia: "NINGUNO", "ROJO"…

Preguntas de la guía

1. ¿Por qué id puede empezar en 0?
Con `autoGenerate` el 0 es un centinela ("sin id todavía"); al insertar, SQLite asigna el siguiente.

2. ¿Qué parte existe para Room y qué parte es del problema?
Para Room: `@Entity`, `@PrimaryKey`, `tableName`, el id autogenerado. Del problema: título, contenido, importante, fecha, color.

3. ¿Guardarías un Painter, un Modifier o un texto formateado?
No: Painter y Modifier son objetos de la UI sin sentido fuera del proceso; un texto formateado impide ordenar y reformatear.

4. ¿Cambiar el nombre de una propiedad puede afectar una base instalada?
Sí: cambia el esquema. Hay que subir la versión y migrar, o fijar el nombre con `@ColumnInfo(name = ...)`.

> **Criterio de finalización**
> ✅ Compila con una `@Entity` válida y PK clara; la explicación de qué representa una fila está en el código.

Ejercicio 3 — DAO: escribir y preguntar a la base

Consigna: Crear `NotaDao`: insertar, listar todas, borrar todas, más una consulta por id y otra de importantes. Desafío: contar sin cargar filas.
Rama: `ej03-dao`

> **En pocas palabras**
> El DAO es la **lista de cosas que se le pueden pedir a la tabla**. Uno escribe la firma y el SQL; Room genera el código. Todavía no se conecta a la pantalla.

> **Aclaraciones para novatos**
> - **El DAO es una `interface`**: no se implementa a mano. KSP genera la clase al compilar.
> - **`@Query` lleva SQL dentro de un String**, y Room lo verifica: si una columna no existe, el proyecto no compila.
> - **`insertar` devuelve un `Long`:** el id que le tocó a la fila nueva.
> - **Buscar por id puede no encontrar nada:** por eso devuelve `NotaEntity?` (puede ser null).
> - **`suspend`** porque leer o escribir en disco puede tardar; Room no deja hacerlo en el hilo principal.

Explicación y pruebas

```
@Dao
interface NotaDao {
    @Insert suspend fun insertar(nota: NotaEntity): Long
    @Query("SELECT * FROM notas ORDER BY creadaEn DESC")
    suspend fun obtenerTodasUnaVez(): List<NotaEntity>
    @Query("DELETE FROM notas") suspend fun borrarTodas()
    @Query("SELECT * FROM notas WHERE id = :id LIMIT 1")
    suspend fun obtenerPorId(id: Int): NotaEntity?            // propia
    @Query("SELECT * FROM notas WHERE importante = 1 ORDER BY creadaEn DESC")
    suspend fun obtenerImportantes(): List<NotaEntity>        // propia
    @Query("SELECT COUNT(*) FROM notas")
    suspend fun contarNotas(): Int                            // desafío
}
```

Preguntas de la guía

1. ¿Qué interpreta Room y qué SQLite?
Room: las anotaciones y la firma (genera código, convierte Entity ⇄ fila, valida el SQL al compilar). SQLite: el texto SQL, que ejecuta contra el archivo.

2. ¿Por qué insertar() devuelve Long?
Devuelve el rowId de la fila insertada, que con PK autogenerada es el id nuevo.

3. ¿Y si la consulta por id no encuentra nada?
No hay fila: el retorno razonable es nullable (`NotaEntity?`) y null significa “no existe”.

4. ¿Por qué no bloquear el hilo principal?
Disco puede tardar: congelaría la UI (ANR). Son `suspend` y Room usa su propio hilo de IO.

Criterio de finalización

✅ 6 operaciones útiles, todas sobre columnas que existen en `NotaEntity` (si no, no compilaría).